Amazon EBS: Elastic Block Store

Volumes, volume types, encryption, resizing, snapshots, Lifecycle Manager and Recycle Bin in simple words, with steps and commands

Part	Topic	One-line idea
1	What is EBS?	A virtual hard drive for EC2
2	Important points	AZ, redundancy, many volumes
3	Volume types	gp3, io2 and others
4	Encryption	Protect data with AES-256
5	Delete on termination	Keep or lose the disk with the instance
6	Lab: create, attach, mount, resize, detach	Step by step with commands
7	Snapshots	Backups, and moving data to another AZ or region
8	Lifecycle Manager and Recycle Bin	Automate backups, undo deletes
9	AWS CLI, cost checklist and quiz	Commands, avoid bills, revise

1. What is EBS?

Amazon EBS (Elastic Block Store) is a **cloud storage service for EC2**. It works like a **virtual hard drive**: the server's disk is not inside the computer, it is connected over the network. When you launch an instance, the **8 GiB gp3** disk in the storage step is an EBS volume.

Easy picture: Think of a **pen drive or portable hard disk**. You save work on it, unplug it from one laptop and plug it into another, and your data is still there. An EBS volume works the same way with EC2 instances.

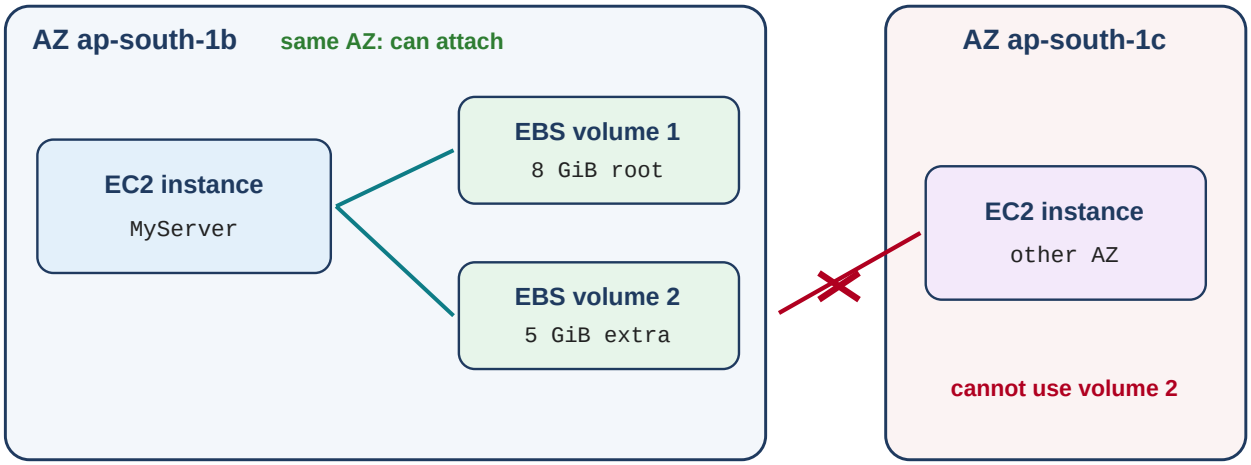
Term	Simple meaning
Volume	One EBS disk.
Attach / Detach	Plug the volume into an instance / unplug it.
Root volume	The disk with the operating system (the instance boots from it).
Snapshot	A backup of a volume.
IOPS	Input/output operations per second: how many reads and writes a disk can do.
Throughput	How much data per second the disk can move (MiB/s).

Why use EBS?

- **Data survives the instance:** if the instance is terminated, the volume can stay (see section 5) and you can attach it to a new instance.
- **Reliable, fast storage** for databases such as MySQL and PostgreSQL.

- **Backups** with snapshots, and **data redundancy** inside the AZ.
- **Many volumes** can be attached to one instance, like adding more hard drives to a laptop.

2. Important Points



Point	Meaning
Region and AZ specific	A volume is created in **one Availability Zone**. An instance can use it only if the instance is in the **same AZ**. (Volume in ap-south-1b, instance in ap-south-1c: not possible.)
Built-in redundancy	Inside its AZ the data is replicated. If one storage block fails, your data is safe.
But not across AZs	If the **whole AZ** fails, the volume is lost. Protect against this with **snapshots**.
Many volumes per instance	You can attach several volumes to one instance.
One instance per volume	A normal volume attaches to one instance at a time. (io1 and io2 support a special Multi-Attach feature.)

Remember: Instance and volume must be in the same AZ. To check, see the **Availability Zone** column in both the Instances list and the Volumes list.

3. Volume Types

There are different kinds of EBS volumes. **Higher performance costs more**, so choose by need.

Type	Kind	Best for
gp3	General purpose SSD (default today). Baseline 3,000 IOPS, and you can raise IOPS and throughput separately.	Most workloads: web servers, small databases, boot volumes
gp2	General purpose SSD (older). Performance grows with size.	Normal operations (gp3 is usually cheaper)
io2 (and io1)	Provisioned IOPS SSD. You pay for the exact IOPS you ask for. Highest speed and durability.	Heavy input/output: large, busy databases
st1	Throughput optimized HDD (magnetic). Cheap for big, sequential data.	Big data, logs, data warehouses

Type	Kind	Best for
sc1	Cold HDD. The cheapest.	Rarely used data

Note: SSD types (gp, io) can be boot volumes. HDD types (st1, sc1) cannot. Limits and prices change, so check the AWS EBS pages before choosing.

4. Encryption

Encryption scrambles your data so only authorised users can read it. In the storage settings (Advanced) choose **Encrypted**. By default a new volume is **not encrypted** unless your account has default encryption turned on.

What is protected	Encrypted?
Data at rest (stored on the volume)	Yes
Data moving between the instance and the volume	Yes (the volume is connected over the network)
Snapshots made from the volume	Yes
Volumes created from those snapshots	Yes

- It uses the strong **AES-256** algorithm, with keys from **AWS KMS** (Key Management Service). You can use the default AWS-managed key or your own.
- An existing unencrypted volume cannot be switched on directly. Make a snapshot, **copy it with encryption enabled**, then create a volume from the copy.

5. Delete on Termination

This setting decides what happens to a volume **when the instance is terminated**.

Setting	Result
Delete on termination = Yes	The volume is **deleted** with the instance. The data is gone.
Delete on termination = No	The volume is **kept**. You can attach it to another instance.

- The **root volume** is **Yes by default**. So terminating a simple instance also deletes its disk.
- To keep it: while launching, open **Configure storage, Advanced**, expand the volume and set **Delete on termination** to **No**.
- To see it later: select the instance, open the **Storage** tab and look at the **Delete on termination** column.
- A volume you attach later to a running instance is kept by default. Always check the setting for volumes added at launch.

Demo: The demo in the video: with the default **Yes**, terminating the instance made the volume disappear from the Volumes list. With **No**, the volume stayed after termination.

6. Lab: Create, Attach, Mount, Resize and Detach a Volume

1. **Launch an instance**
   Name it, keep the defaults (8 GiB gp3). Note its Availability Zone
2. **Create a volume**
   EC2, Elastic Block Store, Volumes, Create volume: 5 GiB, gp3, SAME AZ as the instance
3. **Attach the volume**
   Select it, Actions, Attach volume, choose the instance and a device name, for example /dev/sdb
4. **Check inside the server**
   Connect with SSH and run lsblk: the new disk appears. No reboot needed
5. **Format and mount**
   Create a file system, make a folder, mount the disk (commands below)
6. **Resize (optional)**
   Actions, Modify volume: 5 to 6 GiB. Only increase. Then grow the file system
7. **Unmount, detach and delete**
   umount in Linux, then Detach volume and Delete volume in the console

Note: Device names: you choose **/dev/sdb** in the console. Inside Linux it appears as **/dev/xvdb** on older instance types, or as **/dev/nvme1n1** on newer ones (t3, m5 and later). Use **lsblk** to see the real name, and check the size.

Step 4: see the new disk

COPY AND PASTE: Look at the disks

```
lsblk
```

Output (what students will see)

```
NAME         MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
nvme0n1      259:0    0   8G  0 disk
+-nvme0n1p1  259:1    0   8G  0 part /          <-- root disk, in use
nvme1n1      259:4    0   5G  0 disk            <-- the new 5 GiB volume
```

Step 5: format and mount (a NEW, empty volume)

COPY AND PASTE: Format and mount (replace DEV with your device)

```
DEV=/dev/nvme1n1
sudo file -s $DEV
sudo mkfs -t ext4 $DEV
sudo mkdir -p /data
sudo mount $DEV /data
df -h /data
```

Command	What it does (in simple words)
`DEV=/dev/nvme1n1`	Store the device name once. Use the name you saw in lsblk.
`sudo file -s $DEV`	Shows what is on the disk. **data** means empty, with no file system.
`sudo mkfs -t ext4 $DEV`	Creates a file system. **Only for a new, empty volume. It erases everything.**
`sudo mkdir -p /data`	Makes the folder where the disk will appear (the mount point).
`sudo mount $DEV /data`	Plugs the disk into that folder.
`df -h /data`	Shows the mounted size.

Output (what students will see)

```
/dev/nvme1n1: data              <-- empty disk
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme1n1    4.9G   24K  4.6G   1% /data
```

Make the mount survive a reboot

COPY AND PASTE: Add the disk to /etc/fstab

```
UUID=$(sudo blkid -s UUID -o value $DEV)
echo "UUID=$UUID /data ext4 defaults,nofail 0 2" | sudo tee -a /etc/fstab
sudo mount -a
df -h /data
```

Command	What it does (in simple words)
`UUID=$(sudo blkid -s UUID -o value $DEV)`	Gets the disk's unique ID (more reliable than the device name).
`echo "UUID=... /data ext4 defaults,nofail 0 2" | sudo tee -a /etc/fstab`	Adds a line to the mount list. nofail lets the server boot even if the disk is missing.
`sudo mount -a`	Tests the fstab by mounting everything. No error means it is correct.

Step 6: resize without downtime

In the console: select the volume, **Actions, Modify volume**, change **5** to **6** GiB, click **Modify**. You can only **increase** the size. **No restart** is needed and **no data is lost**. After a minute, Linux sees the larger disk, but the file system must be grown too.

COPY AND PASTE: See the new size and grow the file system (ext4 data disk)

```
lsblk
sudo resize2fs $DEV
df -h /data
```

Command	What it does (in simple words)
`lsblk`	The disk now shows 6G.
`sudo resize2fs $DEV`	Grows an ext4 file system to fill the disk. (For XFS use: sudo xfs_growfs /data.)
`df -h /data`	Shows the larger size.

More: If you resized the **root disk** (which has a partition), first run `sudo growpart /dev/nvme0n1 1`, then `sudo resize2fs /dev/nvme0n1p1` (or `sudo xfs_growfs -d /` for XFS). You can modify a volume again only after a waiting period (about 6 hours). To combine several disks flexibly, use **LVM**.

Step 7: unmount, detach and delete

COPY AND PASTE: Unmount before detaching

```
cd ~
sudo umount /data
df -h | grep /data
sudo sed -i '/ \/data /d' /etc/fstab
```

Command	What it does (in simple words)
`cd ~`	Leave the folder first, otherwise it is busy.
`sudo umount /data`	Unplug the disk in Linux. If it says target is busy, close what uses it or use sudo umount -l /data.
`df -h | grep /data`	Prints nothing: it is no longer mounted.
`sudo sed -i '/ \/data /d' /etc/fstab`	Removes the /data line from fstab (only needed if you added it).

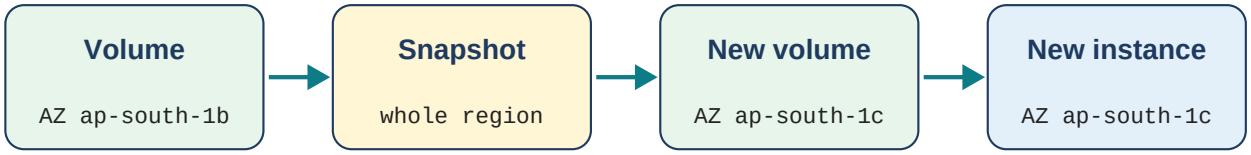
Then in the console: select the volume, **Actions, Detach volume**, wait until the state is **Available**, then **Actions, Delete volume** and type **delete** to confirm. Deleting is permanent. A volume you do not use still costs money.

7. EBS Snapshots

A **snapshot** is a **backup of an EBS volume**. If the volume is corrupted or deleted, you create a new volume from the snapshot. Snapshots can also copy your data to **another AZ or another region**.

Fact	Meaning
Stored in	Amazon S3, managed by AWS, at **region** level (not tied to one AZ).
Incremental	The first snapshot is full. Later ones store only what changed, which saves cost.
Status	Wait until it is **Completed** (not pending or in progress).
Restore	Create a new volume from the snapshot in **any AZ of the region**.
Copy	Copy the snapshot to **another region**, then create a volume there.

Create a snapshot and use it in another AZ



Step	Console action
1	Volumes: select the volume, Actions, Create snapshot. Add a description.
2	Snapshots: wait until the status is Completed.
3	Select the snapshot, Actions, Create volume from snapshot. Choose the new **Availability Zone** (for example ap-south-1c).
4	Launch an instance in that AZ: in Network settings, edit and choose the subnet of that AZ.
5	Volumes: select the new volume, Actions, Attach volume to the new instance. Mount it.

Mount a volume restored from a snapshot

In the video, the snapshot was of an instance's **root volume**, so the restored disk has **partitions**, and the file system is **XFS** (Amazon Linux). Mount the **partition** (**p1**), not the whole disk.

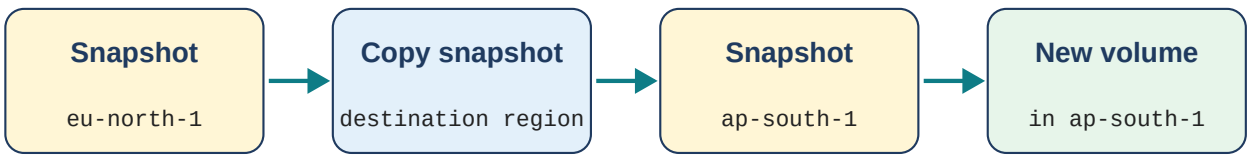
COPY AND PASTE: Mount a restored root-disk volume

```
lsblk
DEV=/dev/nvme1n1
sudo file -s ${DEV}p1
sudo mkdir -p /mnt/mybackup
sudo mount -o nouuid ${DEV}p1 /mnt/mybackup
ls /mnt/mybackup/home
```

Command	What it does (in simple words)
`sudo file -s ${DEV}p1`	Shows the file system of the partition (for example SGI XFS filesystem).
`sudo mkdir -p /mnt/mybackup`	The folder to mount into.
`sudo mount -o nouuid ${DEV}p1 /mnt/mybackup`	**-o nouuid** is needed for XFS: the copy has the same unique ID (UUID) as the running root disk, and a plain mount fails with 'wrong fs type, bad superblock'.
`ls /mnt/mybackup/home`	Your old files (for example my_folder) are there.

Note: If the snapshot was of a **data volume** (no partitions), mount the device directly: `sudo mount $DEV /mnt/mybackup`. On ext4 (Ubuntu) the nouuid option is not needed.

Copy a snapshot to another region



Select the snapshot, **Actions, Copy snapshot**, choose the **destination region** (for example Asia Pacific Mumbai, **ap-south-1**) and click Copy. Switch the console to that region: the snapshot is there (and not in other regions). Then create a volume from it, attach and mount as before. Cross-region copying has data transfer charges.

8. Data Lifecycle Manager and Recycle Bin

Data Lifecycle Manager (automatic snapshots)

Taking snapshots by hand every day is tiring. **Amazon Data Lifecycle Manager** (EBS, Lifecycle Manager) **automates the creation, retention, copying and deletion of snapshots** with a policy. The service itself is **free**. You pay only for the snapshots it makes.

1. **Tag the volume**
   Volume, Manage tags: for example Key = env, Value = test. The policy finds volumes by tag
2. **Open Lifecycle Manager**
   EC2, Elastic Block Store, Lifecycle Manager, Create lifecycle policy
3. **Choose Custom policy**
   EBS snapshot policy. Resource type: Volume. Target tag: env = test
4. **Set the schedule**
   Name, frequency (for example every 12 hours or daily) and start time
5. **Set retention**
   Keep by count (last N) or by age (for example 10 days). Older snapshots are deleted automatically
6. **Optional extras**
   Cross-region copy (extra charges), tags, fast restore
7. **Review and create**
   Use the default role. State: Enabled

Recycle Bin (undo a delete)

Snapshots can be deleted by mistake. The **Recycle Bin** keeps deleted snapshots for a while so that you can **restore** them, like the Recycle Bin on a desktop. You create a **retention rule**: the resource type (EBS snapshots) and how many days to keep deleted items (for example 7 days). Deleted snapshots that match the rule appear in the Recycle Bin and can be recovered within that time. Storage in the Recycle Bin is billed like normal snapshots.

	Data Lifecycle Manager	Recycle Bin
Purpose	Create and delete snapshots automatically on a schedule	Recover snapshots deleted by accident
Key setting	Schedule and retention (count or days)	Retention rule (days to keep deleted items)

	Data Lifecycle Manager	Recycle Bin
Cost	Service free; pay for snapshots	Pay for stored snapshots

9. AWS CLI, Cost Checklist and Quiz

Optional: the same actions with the AWS CLI (run in your terminal after `aws configure`, or in AWS CloudShell). Replace the IDs.

COPY AND PASTE: EBS with the AWS CLI

```
aws ec2 create-volume --availability-zone ap-south-1b --size 5 \
  --volume-type gp3
aws ec2 attach-volume --volume-id VOL_ID --instance-id INSTANCE_ID \
  --device /dev/sdb
aws ec2 modify-volume --volume-id VOL_ID --size 6
aws ec2 create-snapshot --volume-id VOL_ID --description "test snapshot"
aws ec2 create-volume --availability-zone ap-south-1c --snapshot-id SNAP_ID
aws ec2 copy-snapshot --source-region eu-north-1 --source-snapshot-id SNAP_ID \
  --region ap-south-1
```

Command	What it does (in simple words)
`create-volume`	Creates a volume in a chosen AZ (size in GiB). With --snapshot-id it is built from a snapshot.
`attach-volume`	Attaches it to an instance in the same AZ.
`modify-volume --size 6`	Increases the size (Elastic Volumes).
`create-snapshot`	Backs up a volume.
`copy-snapshot`	Copies a snapshot into the region given by --region (the destination).

Cost checklist: (1) **Delete volumes you no longer need**: you pay for the size you created, even when it is unused. (2) **Delete old snapshots**, or let a Lifecycle policy do it. (3) Check **Delete on termination** so you do not keep forgotten volumes, and also do not lose data you need. (4) Cross-region copies cost extra. (5) Terminate practice instances and delete test volumes.

Question	Answer
What is EBS?	A cloud block storage service: a virtual hard drive for EC2.
Can an instance in AZ c use a volume in AZ b?	No. They must be in the same AZ.
What if the whole AZ fails?	The volume is lost, unless you have snapshots.
Which type for heavy database I/O? For general use?	io2 (Provisioned IOPS) / gp3.
Can you shrink an EBS volume?	No. You can only increase the size.
Do you need to restart to resize or attach?	No.

Question	Answer
What does Delete on termination = No do?	The volume stays after the instance is terminated.
How do you move a volume to another AZ?	Snapshot, then create a volume from it in the new AZ.
How do you move data to another region?	Copy the snapshot to that region, then create a volume.
Why mount with -o nouuid?	A restored XFS disk has the same UUID as the root disk.
Which service automates snapshots? Which undoes deletes?	Data Lifecycle Manager / Recycle Bin.

- EBS = a virtual hard drive for EC2, in one AZ, with built-in redundancy.
- Snapshots are the backup tool: restore, move to another AZ, copy to another region.
- Lifecycle Manager automates snapshots, Recycle Bin protects them from accidental deletion.